

# **Sistem de tranzactionare algoritmica folosind Logistic Regression si Q-learning**

David Gavrilescu

5 iunie 2026

## **Rezumat**

Lucrarea prezinta un sistem simplificat de tranzactionare algoritmica care combina Logistic Regression si Q-learning. Modelul ML produce un semnal despre directia pietei, iar agentul RL foloseste acel semnal pentru a decide cand sa cumpere, sa vanda sau sa astepte. Rezultatele sunt mixte si scopul proiectului este academic, nu construirea unei strategii reale de investitii.

# Cuprins

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>Rezumat</b>                                            | <b>1</b>  |
| <b>1 Introducere</b>                                      | <b>3</b>  |
| 1.1 Instrumentele financiare folosite . . . . .           | 3         |
| 1.2 Ideea sistemului . . . . .                            | 4         |
| <b>2 Datele folosite</b>                                  | <b>4</b>  |
| <b>3 Arhitectura aplicatiei</b>                           | <b>5</b>  |
| <b>4 Tehnologii utilizate</b>                             | <b>6</b>  |
| <b>5 Prelucrarea datelor</b>                              | <b>7</b>  |
| <b>6 Modelul Logistic Regression</b>                      | <b>8</b>  |
| <b>7 Agentul Q-learning</b>                               | <b>10</b> |
| <b>8 Backtesting si strategii comparate</b>               | <b>12</b> |
| <b>9 Rezultate</b>                                        | <b>15</b> |
| 9.1 Rezultate pe SPY . . . . .                            | 15        |
| 9.2 Rezultate pe QQQ . . . . .                            | 16        |
| 9.3 Analiza politicii Q-learning in starea cash . . . . . | 16        |
| 9.4 Interpretarea generala . . . . .                      | 17        |
| <b>10 State of the Art si pozitionarea proiectului</b>    | <b>17</b> |
| 10.1 Context . . . . .                                    | 17        |
| 10.2 Lucrarea Theate si Ernst . . . . .                   | 17        |
| 10.3 Pozitionarea proiectului . . . . .                   | 18        |
| 10.4 Comparatie orientativa cu TDQN . . . . .             | 18        |
| <b>11 Limitari</b>                                        | <b>19</b> |
| <b>12 Manual de utilizare</b>                             | <b>20</b> |
| <b>13 Concluzii</b>                                       | <b>22</b> |

# 1 Introducere

Proiectul construiește un sistem simplificat de tranzactionare algoritmica pe ETF-urile SPY si QQQ. Pe scurt, sistemul primește date istorice de piata si incearca sa aleaga automat intre trei actiuni: cumpara, vinde sau nu face nimic.

Am impartit problema in doua parti. Prima parte este de Machine Learning si incearca sa obtina un semnal despre directia pietei. A doua parte este de Reinforcement Learning si folosește acel semnal pentru decizia finala. Practic, Logistic Regression nu tranzactioneaza singur, ci doar ii da agentului Q-learning o informatie in plus.

Scopul proiectului este academic. Nu incerc sa construiesc un bot real de trading si nici sa demonstrez ca sistemul bate piata. De fapt, rezultatele arata destul de clar ca lucrrurile depind mult de instrumentul testat: pe SPY rezultatul este bun, pe QQQ este mai defensiv, dar nu bate Buy and Hold la randament.

**Tabelul 1:** Termeni tehnici folositi in lucrare

| Termen              | Explicatie scurta                                                                      |
|---------------------|----------------------------------------------------------------------------------------|
| ETF                 | Fond tranzactionat la bursa. In proiect sunt folosite SPY si QQQ                       |
| OHLCV               | Date zilnice de piata: Open, High, Low, Close si Volume                                |
| Feature             | O caracteristica folosita de model, de exemplu randament zilnic sau schimbare de volum |
| Label               | Raspunsul corect folosit la antrenare. In proiect este 0 sau 1                         |
| Train               | Partea de date pe care modelul invata                                                  |
| Test                | Partea de date pe care modelul nu o vede la antrenare si pe care este verificat        |
| Logistic Regression | Model simplu de ML care produce o probabilitate intre 0 si 1                           |
| Semnal ML           | Probabilitatea transformata in trei valori: negativ, neutru sau pozitiv                |
| Q-learning          | Algoritm RL care invata ce actiune este mai buna intr-o anumita stare                  |
| Q-table             | Tabela in care agentul tine valorile invatate pentru fiecare stare si actiune          |
| Reward              | Recompensa primita de agent dupa o actiune                                             |
| Politica            | Regula invatata de agent: ce actiune alege in fiecare stare                            |
| Backtesting         | Testarea strategiei pe date istorice, ca si cum ar fi fost folosita atunci             |
| Buy and Hold        | Strategie simpla: cumpara la inceput si pastreaza pana la final                        |
| Sharpe Ratio        | Indicator care compara randamentul cu riscul                                           |
| Max Drawdown        | Cea mai mare scadere de la un maxim al portofoliului pana la un minim                  |

## 1.1 Instrumentele financiare folosite

Inainte de descrierea sistemului, este util sa fie clarificat tipul de date folosit in experiment.

Un ETF este un instrument financiar care poate fi cumparat si vandut la bursa, similar unei actiuni obisnuite. Diferenta fata de o actiune individuala este ca un ETF nu reprezinta o singura companie, ci urmareste un grup de companii, un sector sau un indice. Din acest motiv, un ETF poate fi mai reprezentativ pentru evolutia unei piete decat o actiune izolata.

Experimentul principal folosește SPY. SPY este un ETF care urmareste indicele S&P 500, adica un indice format din companii mari listate pe pietele americane [2]. Am ales SPY

deoarece are istoric lung de date, lichiditate mare si este bine documentat, deci este potrivit pentru un proiect academic.

Ca verificare suplimentara, am testat acelasi sistem si pe QQQ. QQQ este un ETF care urmareste Nasdaq-100, un indice orientat mai mult spre companii mari din zona tehnologiei [3]. QQQ nu reprezinta experimentul principal, ci il folosesc pentru a observa daca sistemul are un comportament similar si pe un instrument financiar cu alte caracteristici.

**Tabelul 2:** Instrumentele financiare folosite in proiect

| Instrument | Rol in proiect          | Descriere                               |
|------------|-------------------------|-----------------------------------------|
| SPY        | Experiment principal    | ETF care urmareste indicele S&P 500.    |
| QQQ        | Verificare suplimentara | ETF care urmareste indicele Nasdaq-100. |

## 1.2 Ideea sistemului

Sistemul are doua parti principale. Prima parte transforma datele de piata in feature-uri si antreneaza Logistic Regression. A doua parte foloseste semnalul produs de Logistic Regression intr-un agent Q-learning.

Logistic Regression nu face tranzactii direct. El produce o probabilitate, iar probabilitatea este transformata intr-un semnal simplu: negativ, neutru sau pozitiv. Q-learning primeste acest semnal, pozitia curenta si trendul, apoi alege actiunea finala: BUY, SELL sau HOLD.

## 2 Datele folosite

Datele folosite vin din setul public *S&P 500 and NASDAQ 100: Daily Data*, disponibil pe Kaggle [1]. Din acest set am folosit fisierele `SPY.csv` si `QQQ.csv`. Am ales fisiere CSV locale, nu descarcare automata, ca rezultatele sa fie reproductibile.

Setul contine date zilnice de tip OHLCV:

- **Open** – pretul de deschidere
- **High** – pretul maxim al zilei
- **Low** – pretul minim al zilei
- **Close** – pretul de inchidere
- **Volume** – volumul tranzactionat
- **Adjusted Close** – pretul ajustat pentru dividende si split-uri

Perioada folosita este 2010-01-01 – 2019-01-01. Data de final nu este inclusa. Dupa calcularea mediilor mobile si a randamentului viitor pe 5 zile, raman 2240 de randuri.

**Tabelul 3:** Setul de date folosit

| Element                 | Detalii                                           |
|-------------------------|---------------------------------------------------|
| Sursa                   | Kaggle – <i>SP 500 and NASDAQ 100: Daily Data</i> |
| Fisiere                 | SPY.csv, QQQ.csv                                  |
| Perioada                | 2010-01-01 – 2019-01-01                           |
| Randuri dupa curatare   | 2240                                              |
| Train                   | 2010-02-01 – 2016-04-22, 1568 randuri             |
| Test                    | 2016-04-25 – 2018-12-21, 672 randuri              |
| Experiment principal    | SPY                                               |
| Verificare suplimentara | QQQ                                               |

Impartirea datelor este cronologica: primele 70% pentru antrenare, ultimele 30% pentru testare.

### 3 Arhitectura aplicatiei

Structura proiectului este simpla. Fiecare fisier are un rol clar, iar `main.py` le leaga pe toate intr-o singura rulare.

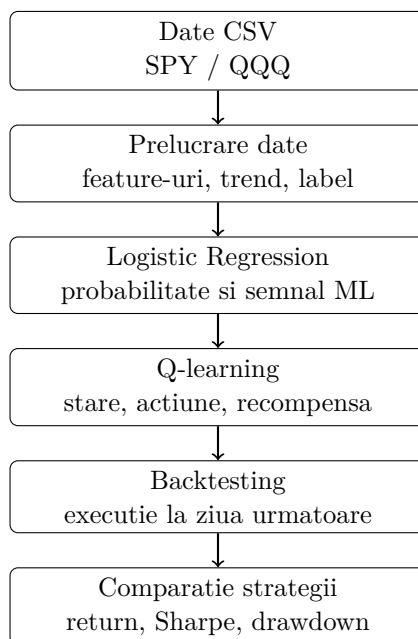
**Tabelul 4:** Structura proiectului

| Fisier                              | Rol                                                                                         |
|-------------------------------------|---------------------------------------------------------------------------------------------|
| <code>config.py</code>              | Contine setarile principale: ticker, perioada, cost tranzactie, parametri LR si Q-learning. |
| <code>data_processing.py</code>     | Incarca datele CSV, ajusteaza preturile si calculeaza feature-urile.                        |
| <code>train_test.py</code>          | Face split cronologic, adauga label-ul pentru ML si normalizeaza feature-urile.             |
| <code>logistic_regression.py</code> | Antreneaza manual Logistic Regression si produce probabilitati.                             |
| <code>ml_signal.py</code>           | Transforma probabilitatea LR in semnal discret: 0, 1 sau 2.                                 |
| <code>q_learning.py</code>          | Defineste starile, actiunile, recompensa si antreneaza Q-table-ul.                          |
| <code>benchmark.py</code>           | Ruleaza strategiile si calculeaza randament, Sharpe Ratio, drawdown si tranzactii.          |
| <code>afisare.py</code>             | Afiseaza rezultatele principale in consola.                                                 |
| <code>main.py</code>                | Ruleaza pipeline-ul complet.                                                                |

Fluxul de rulare este urmatorul:

1. se incarca datele pentru tickerul ales

2. se calculeaza feature-urile
3. se face split cronologic 70% train si 30% test
4. se antreneaza Logistic Regression pe train
5. probabilitatile sunt transformate in semnal ML
6. se antreneaza Q-learning folosind semnalul ML, pozitia si trendul
7. se ruleaza backtestul pe test
8. se compara rezultatele cu strategiile de baza



**Figura 1:** Fluxul principal al aplicatiei.

## 4 Tehnologii utilizate

Implementarea este realizata in Python 3. Am ales Python pentru ca este simplu de folosit pentru date tabelare si calcule numerice, iar pentru un proiect de ML si RL este suficient de clar.

Pentru calcule folosesc **numpy**. Aici intra operatiile cu vectori si matrici, antrenarea Logistic Regression si actualizarea valorilor din tabela Q.

Pentru date folosesc **pandas**. Cu **pandas** citesc fisierele CSV, calculez feature-urile, fac impartirea cronologica in train si test si organizez rezultatele finale in tabele.

Pentru grafice folosesc **matplotlib**. Graficele ajuta mai ales la interpretare, pentru ca din tabelul final nu se vede mereu cum s-a comportat portofoliul pe parcurs.

Datele sunt citite din fisiere CSV locale. Am ales varianta asta ca sa pot reproduce aceleasi rezultate la fiecare rulare, fara sa depind de o descarcare noua.

Raportul este scris in LaTeX, in principal pentru formule, tabele si figuri.

## 5 Prelucrarea datelor

Datele brute nu sunt trimise direct in model. Din ele se calculeaza cateva feature-uri simple, folosind doar informatii disponibile pana in ziua curenta.

**Tabelul 5:** Feature-urile folosite de Logistic Regression

| Feature                        | Explicatie                                                                    |
|--------------------------------|-------------------------------------------------------------------------------|
| <code>daily_return</code>      | Randamentul fata de ziua precedenta                                           |
| <code>open_close_return</code> | Miscarea din aceeasi zi, de la Open la Close                                  |
| <code>high_low_range</code>    | Diferenta dintre High si Low raportata la Close                               |
| <code>volume_change</code>     | Schimbarea volumului fata de ziua precedenta                                  |
| <code>ma_ratio</code>          | Raportul dintre media mobila pe 5 zile si media mobila pe 20 de zile, minus 1 |

Formulele principale sunt scrise mai simplu mai jos. Aici `Close` inseamna pretul zilei curente, iar `Close_prev` inseamna pretul din ziua precedenta.

$$daily\_return = \frac{Close}{Close\_prev} - 1$$

$$open\_close\_return = \frac{Close}{Open} - 1$$

$$high\_low\_range = \frac{High - Low}{Close}$$

$$volume\_change = \frac{Volume}{Volume\_prev} - 1$$

Pentru trend se folosesc doua medii mobile. `MA(5)` este media pe ultimele 5 zile, iar `MA(20)` este media pe ultimele 20 de zile:

$$ma\_ratio = \frac{MA(5)}{MA(20)} - 1$$

Trendul este discretizat cu pragul 0.005:

$$trend = \begin{cases} 0, & ma\_ratio < -0.005 \\ 1, & -0.005 \leq ma\_ratio \leq 0.005 \\ 2, & ma\_ratio > 0.005 \end{cases}$$

Pentru Logistic Regression se calculeaza si randamentul peste 5 zile:

$$future\_return\_5d = \frac{Close\_after\_5\_days}{Close} - 1$$

Aceasta coloana nu intra ca feature in model. Este folosita doar pentru label:

$$label = \begin{cases} 1, & future\_return\_5d > median\_train \\ 0, & future\_return\_5d \leq median\_train \end{cases}$$

Normalizarea se face cu minimele si maximele din train:

$$x\_norm = \frac{x - x\_min\_train}{x\_max\_train - x\_min\_train}$$

## 6 Modelul Logistic Regression

Logistic Regression este componenta de Machine Learning. Modelul nu prezice pretul exact, ci estimeaza probabilitatea ca label-ul definit anterior sa fie 1.

Modelul calculeaza un scor liniar:

$$z = w_1x_1 + w_2x_2 + \dots + w_5x_5 + b$$

Apoi scorul este transformat intr-o probabilitate cu functia sigmoid:

$$g(z) = \frac{1}{1 + e^{-z}}$$

Probabilitatea este transformata intr-un semnal discret pentru Q-learning:

$$ml\_signal = \begin{cases} 0, & g(z) < 0.49 \\ 1, & 0.49 \leq g(z) \leq 0.51 \\ 2, & g(z) > 0.51 \end{cases}$$

Semnalul 0 inseamna negativ, 1 inseamna neutru, iar 2 inseamna pozitiv. Modelul ML nu executa tranzactii direct. El doar ofera agentului Q-learning un context simplificat.

**Tabelul 6:** Rezultatele Logistic Regression

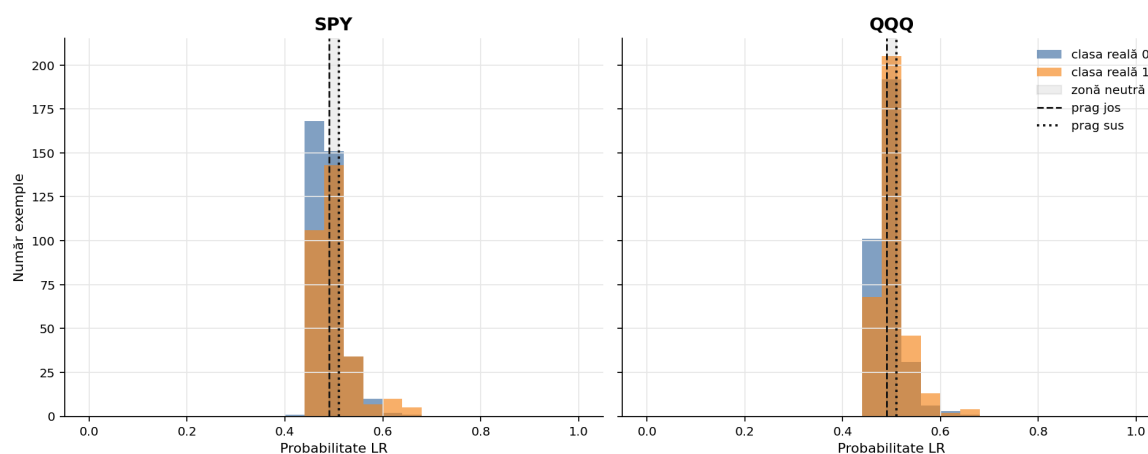
| Indicator          | SPY      | QQQ      |
|--------------------|----------|----------|
| Prag mediana train | 0.004515 | 0.005606 |
| Baseline train     | 50.00%   | 50.00%   |
| Baseline test      | 54.61%   | 50.30%   |
| Acuratete train    | 53.57%   | 53.95%   |
| Acuratete test     | 58.33%   | 53.57%   |
| Bias               | 0.474098 | 0.577429 |



Acuratetea nu este foarte mare, dar nici nu ma asteptam sa fie foarte mare pe date financiare. Piata are mult zgomot, iar modelul foloseste doar cinci feature-uri simple. Pentru proiect, modelul este util mai ales pentru ca produce un semnal care poate fi folosit mai departe de Q-learning, nu pentru ca ar fi un predictor foarte puternic singur.

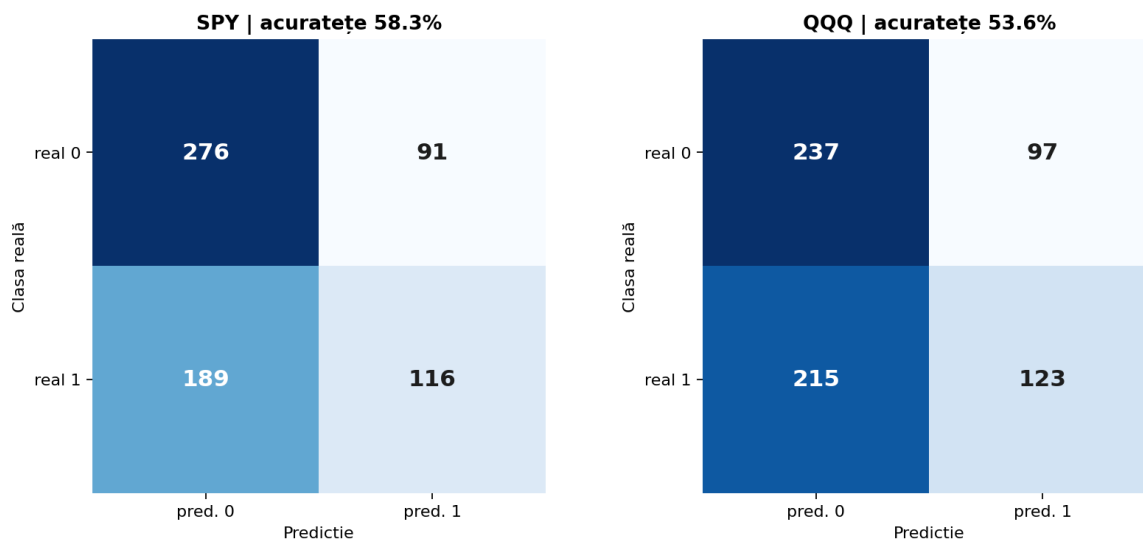
**Tabelul 7:** Ponderile Logistic Regression

| Feature normalizat     | SPY       | QQQ       |
|------------------------|-----------|-----------|
| daily_return_norm      | -0.312674 | -0.331951 |
| open_close_return_norm | -0.227683 | -0.423075 |
| high_low_range_norm    | 0.953636  | 0.720454  |
| volume_change_norm     | 0.416079  | 0.158979  |
| ma_ratio_norm          | -0.552194 | -0.433881 |



**Figura 2:** Histograma probabilitatilor produse de Logistic Regression pe perioada de test si pragurile 0.49 / 0.51 folosite pentru semnalul ML.

Graficul arata ca probabilitatile sunt concentrate in jurul valorii 0.5. Modelul nu este foarte sigur pe predictii, lucru asteptat pentru date financiare. Pragurile 0.49 si 0.51 transforma probabilitatea intr-un semnal simplu pentru Q-learning.



**Figura 3:** Matricea de confuzie pe test pentru Logistic Regression, comparativ pentru SPY si QQQ.

Matricea de confuzie arata unde greseste si unde nimereste modelul. Pe ambele ETF-uri, modelul pare sa recunoasca mai bine clasa 0 decat clasa 1. Asta explica partial de ce acuratetea ramane moderata. Modelul ML singur nu este suficient pentru o strategie buna, dar poate da un semnal util pentru agentul Q-learning.

## 7 Agentul Q-learning

Q-learning este componenta de Reinforcement Learning. Agentul invata ce actiune este mai buna intr-o anumita stare. In loc sa primeasca toate preturile brute, agentul primeste o stare discreta.

Starea este:

$$state = (ml\_signal, position, trend)$$

unde:

- `ml_signal` poate fi 0, 1 sau 2
- `position` poate fi 0 pentru cash sau 1 pentru detinere ETF
- `trend` poate fi 0, 1 sau 2

Numarul total de stari este:

$$3 \times 2 \times 3 = 18$$

Pentru a salva starea intr-o tabela, folosesc indexarea:

$$ml\_signal \times 6 + position \times 3 + trend$$

Actiunile posibile sunt BUY, SELL si HOLD, dar actiunile valide depind de pozitie:

$$A(position) = \begin{cases} \{BUY, HOLD\}, & position = 0 \\ \{SELL, HOLD\}, & position = 1 \end{cases}$$

Daca agentul este in cash, nu are ce sa vanda. Daca detine ETF-ul, poate sa vanda sau sa astepte. In proiect nu este folosit short selling.

Recompensa este zilnica. Daca agentul este investit, primeste randamentul zilei. Daca este in cash, recompensa este 0. Pentru BUY si SELL se scade costul de tranzactie:

$$reward = \begin{cases} daily\_return \times 100 - cost, & position = 1 \text{ si se tranzactioneaza} \\ daily\_return \times 100, & position = 1 \text{ si se alege HOLD} \\ -cost, & position = 0 \text{ si se alege BUY} \\ 0, & position = 0 \text{ si se alege HOLD} \end{cases}$$

Agentul primeste miscarea reala a pietei, exprimata pe scala procentuala. Asta apropie antrenarea de backtest, dar face politica mai sensibila la zile cu miscari mari.

Actualizarea Q-learning foloseste formula:

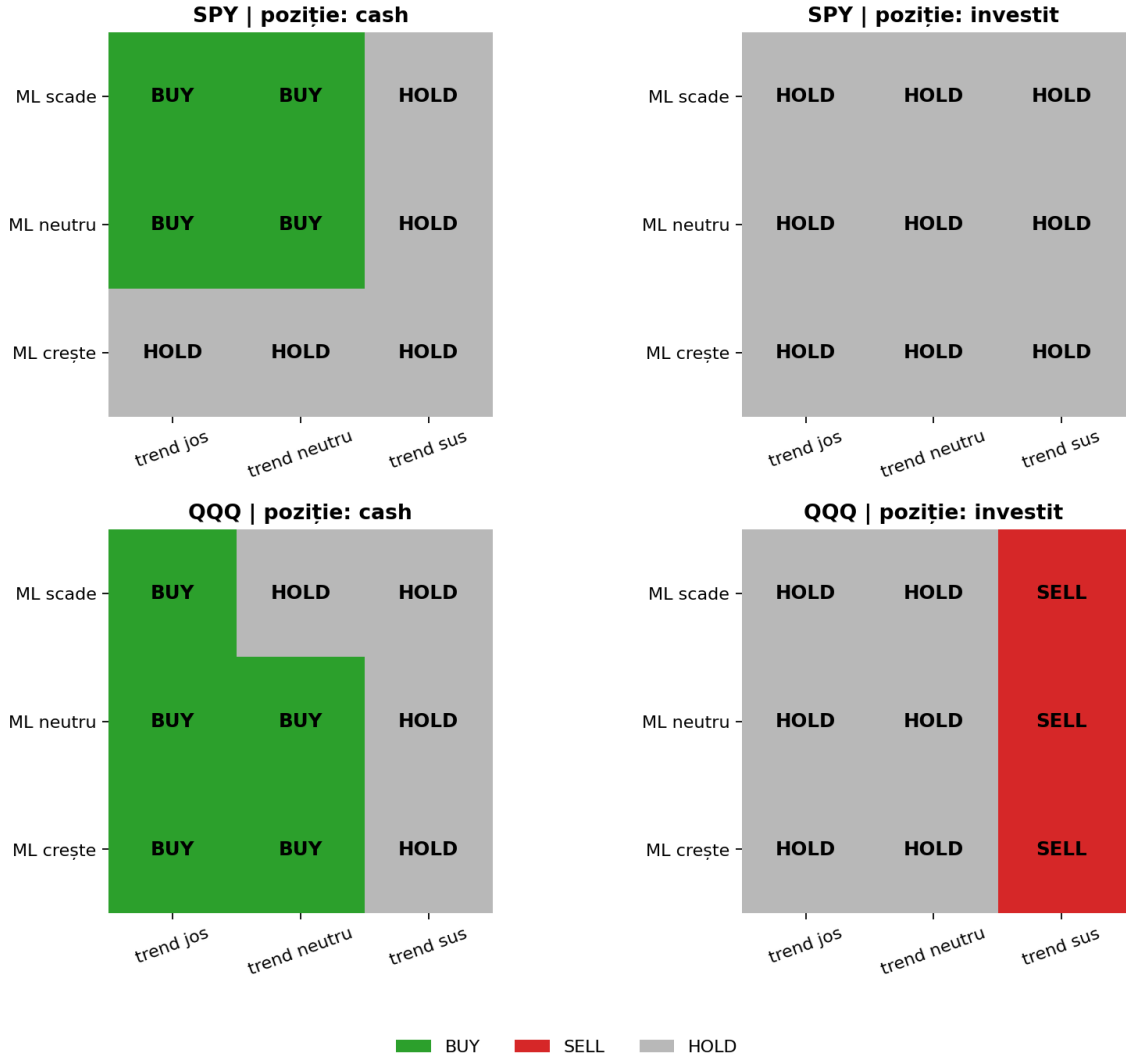
$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[ reward + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Intuitiv, agentul muta valoarea veche spre valoarea noua observata. Aici s este starea curenta, a este actiunea, iar s' este starea urmatoare. Daca actiunea a dus la reward bun, valoarea ei creste. Daca a dus la rezultat slab, valoarea scade.

**Tabelul 8:** Parametrii Q-learning

| Parametru                   | Valoare |
|-----------------------------|---------|
| Episoade                    | 500     |
| Rata de invatare $\alpha$   | 0.1     |
| Factor de discount $\gamma$ | 0.9     |
| Epsilon initial             | 0.3     |
| Epsilon minim               | 0.01    |
| Decay epsilon               | 0.993   |
| Cost tranzactie             | 0.05%   |
| Seed random                 | 42      |

In timpul antrenarii se foloseste epsilon-greedy: la inceput agentul exploreaza mai mult, iar spre final foloseste mai mult ce a invatat. Q-table-ul porneste de la zero. In caz de egalitate intre actiuni, se prefera HOLD, ca sa nu faca tranzactii inutile.



**Figura 4:** Politica invatata de agentul Q-learning. Fiecare celula arata actiunea cu valoarea Q maxima pentru combinatia dintre semnalul ML, pozitie si trend.

Graficul arata politica finala invatata de agent. Pe SPY, m-a surprins putin faptul ca agentul ajunge sa stea aproape mereu in HOLD dupa ce este deja investit. Practic, politica devine apropiata de Buy and Hold. Pe QQQ, agentul este mai activ: cumpara in mai multe stari cash si vinde in unele stari cand este deja investit. Nu interpretez aceste reguli ca reguli generale de piata, ci doar ca rezultat al valorilor Q invatate pe perioada de antrenare.

## 8 Backtesting si strategii comparate

Backtestul verifica politica invatata pe perioada de test. Toate strategiile pornesc de la acelasi capital initial:

$$capital_0 = 100000$$

La fiecare zi din test, strategia decide ce pozitie are pentru ziua urmatoare. Decizia calculata in ziua  $t$  este executata la pretul din ziua  $t + 1$ , ca sa nu folosim informatie din aceeasi zi in

mod nerealist. Nu se foloseste short selling. Strategia poate fi in doua stari: cash sau investita in ETF.

La BUY, portofoliul intra in ETF si se scade costul de tranzactie. La SELL, pozitia este vanduta si portofoliul revine in cash. La HOLD, pozitia ramane neschimbata. Costul de tranzactie folosit in test este acelasi ca in antrenare, adica 0.05%.

Strategiile comparate sunt:

**Tabelul 9:** Strategiile testate in backtest

| Strategie          | Cum functioneaza in test                                                                            |
|--------------------|-----------------------------------------------------------------------------------------------------|
| Buy and Hold       | Cumpara la inceputul testului si pastreaza pozitia pana la final                                    |
| LR only            | Foloseste direct semnalul Logistic Regression: cumpara la semnal pozitiv si vinde la semnal negativ |
| Random Agent avg   | Alege actiuni aleatorii, iar rezultatul este media pe 30 de rulari                                  |
| ML + Q-learning    | Foloseste politica invatata din starea ( <i>ml_signal, position, trend</i> )                        |
| Q-learning fara ML | Foloseste Q-learning doar cu pozitia si trendul, fara semnalul Logistic Regression                  |

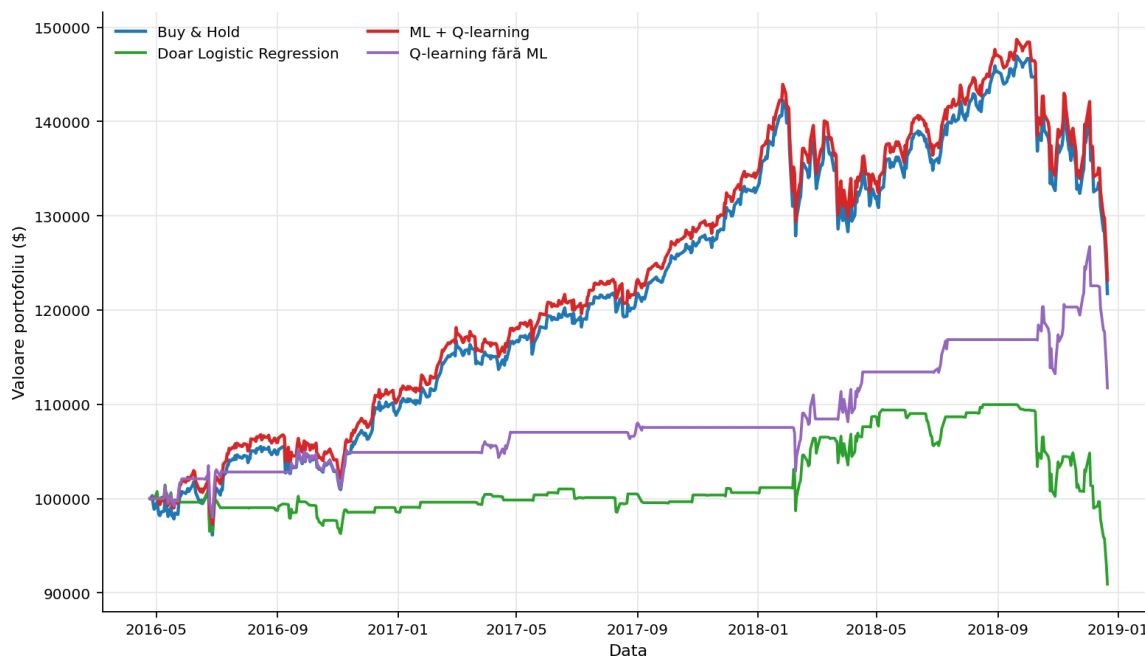
Valoarea portofoliului este actualizata zilnic. Randamentul final este calculat fata de capitalul initial:

$$Return = \left( \frac{capital_{final}}{capital_0} - 1 \right) \times 100$$

Expunerea arata cat timp strategia a stat investita in ETF:

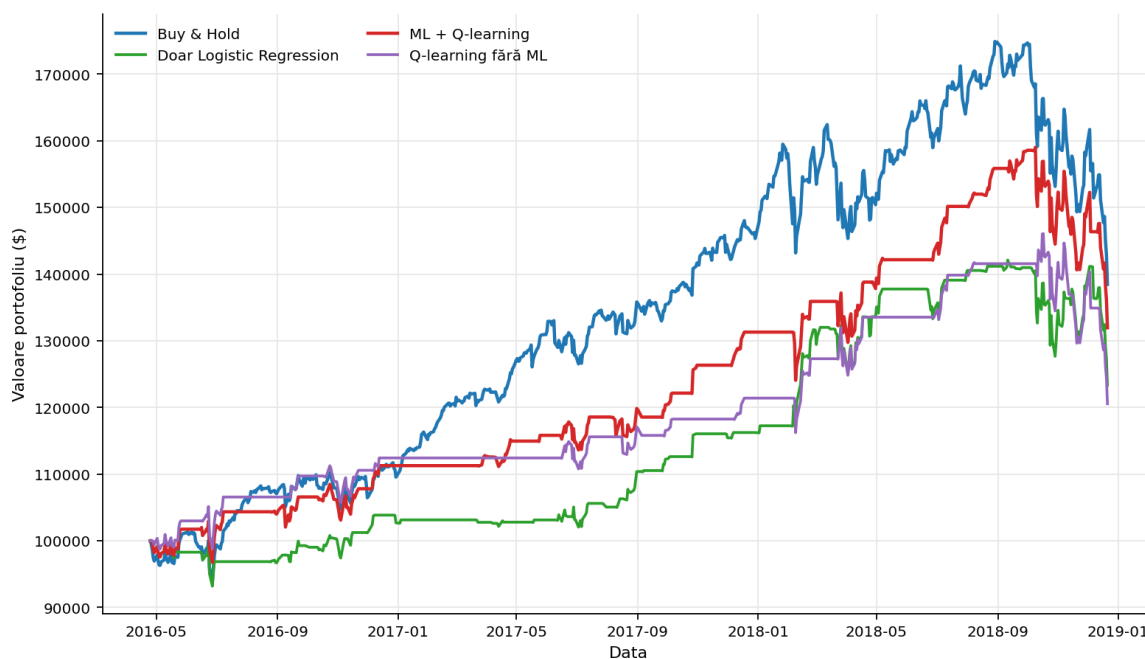
$$Exposure = \frac{zile\_investit}{zile\_test} \times 100$$

Pentru risc se folosesc Sharpe Ratio si Max Drawdown. Sharpe Ratio compara randamentul zilnic mediu cu variatia lui, iar Max Drawdown arata cea mai mare scadere fata de un maxim anterior al portofoliului.



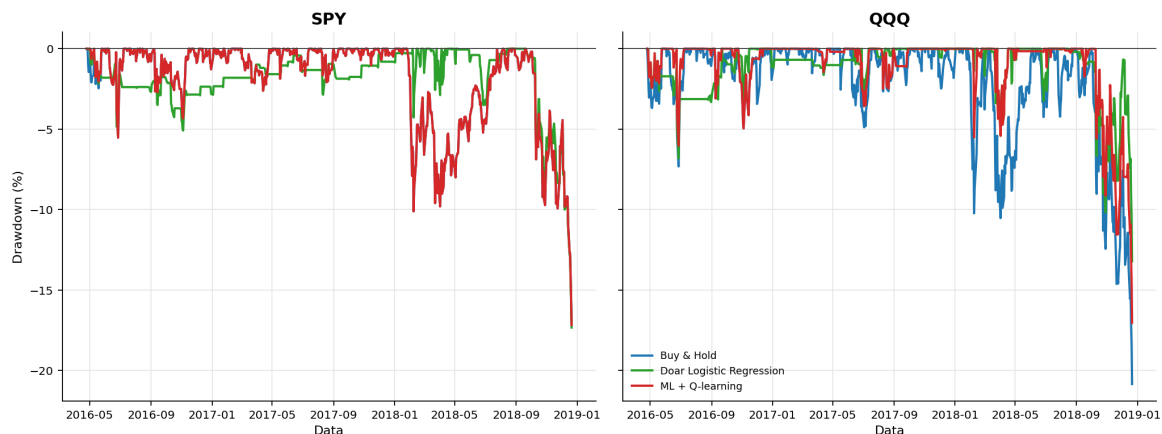
**Figura 5:** Evolutia portofoliului pe perioada de test pentru strategiile principale pe SPY.

Pe SPY, ML + Q-learning urmareste aproape aceeasi directie ca Buy and Hold si ajunge putin peste aceasta la final. Asta se leaga de expunerea foarte mare din tabelul de rezultate: agentul sta investit aproape toata perioada.



**Figura 6:** Evolutia portofoliului pe perioada de test pentru strategiile principale pe QQQ.

Pe QQQ, Buy and Hold ramane cea mai buna strategie dupa randament final. ML + Q-learning ramane profitabila si depaseste variantele mai simple, dar nu depaseste strategia pasiva.



**Figura 7:** Drawdown-ul strategiilor principale pe perioada de test, comparativ cu Buy and Hold.

Graficul de drawdown arata scaderile fata de maximele anterioare ale portofoliului. Pe QQQ, ML + Q-learning are drawdown mai mic decat Buy and Hold, dar si randament final mai mic. De aceea rezultatul nu trebuie judecat doar dupa valoarea finala.

## 9 Rezultate

### 9.1 Rezultate pe SPY

**Tabelul 10:** Benchmark strategii pe SPY

| Strategie          | Valoare   | Rand. % | Tranz. | Exp. % | Sharpe | MDD % |
|--------------------|-----------|---------|--------|--------|--------|-------|
| Buy and Hold       | 121745.02 | 21.75   | 1.00   | 100.00 | 0.68   | 17.16 |
| LR only            | 90913.29  | -9.09   | 135.00 | 27.38  | -0.34  | 17.32 |
| Random Agent avg   | 89789.12  | -10.21  | 336.27 | 49.27  | -0.44  | 16.92 |
| ML + Q-learning    | 123192.36 | 23.19   | 1.00   | 99.11  | 0.72   | 17.16 |
| Q-learning fara ML | 111731.59 | 11.73   | 21.00  | 26.64  | 0.53   | 11.84 |

Pe SPY, strategia ML + Q-learning obtine cel mai bun rezultat numeric: 23.19%, fata de 21.75% pentru Buy and Hold. Diferenta este pozitiva, dar nu foarte mare. Spre deosebire de o strategie care evita mult piata, aici ML + Q-learning sta investit aproape toata perioada de test, cu expunere de 99.11%.

Acest lucru se vede si in equity curve: ML + Q-learning se misca foarte apropiat de Buy and Hold. Drawdown-ul maxim este acelasi cu Buy and Hold, 17.16%, deci avantajul pe SPY este mai ales la randamentul final, nu la reducerea riscului.

Q-learning fara ML obtine 11.73%, deci are rezultat pozitiv, dar ramane clar sub ML + Q-learning. Acest lucru sugereaza ca semnalul Logistic Regression adauga informatie utila peste trend si pozitie, chiar daca strategia finala ramane apropiata de Buy and Hold.

## 9.2 Rezultate pe QQQ

**Tabelul 11:** Benchmark strategii pe QQQ

| Strategie          | Valoare   | Rand. % | Tranz. | Exp. % | Sharpe | MDD % |
|--------------------|-----------|---------|--------|--------|--------|-------|
| Buy and Hold       | 138454.97 | 38.45   | 1.00   | 100.00 | 0.83   | 20.83 |
| LR only            | 123339.50 | 23.34   | 107.00 | 33.93  | 0.69   | 13.21 |
| Random Agent avg   | 97113.95  | -2.89   | 336.27 | 49.27  | -0.06  | 18.81 |
| ML + Q-learning    | 131935.97 | 31.94   | 45.00  | 43.30  | 0.84   | 17.02 |
| Q-learning fara ML | 120557.67 | 20.56   | 31.00  | 29.76  | 0.63   | 17.47 |

Pe QQQ, Buy and Hold ramane strategia cu cel mai mare randament final, 38.45%. ML + Q-learning obtine 31.94%, deci nu depaseste strategia pasiva pe acest ETF. Totusi, ML + Q-learning ramane o strategie profitabila si are Max Drawdown mai mic decat Buy and Hold: 17.02% fata de 20.83%.

Q-learning fara ML obtine 20.56%, iar Doar Logistic Regression obtine 23.34%. Sistemul complet ML + Q-learning este peste aceste variante izolate, ceea ce arata ca integrarea celor doua componente aduce informatie utila, chiar daca nu este suficienta pentru a depasi Buy and Hold pe QQQ.

## 9.3 Analiza politicii Q-learning in starea cash

Pentru a intelege mai clar ce a invatat agentul, am analizat diferenta dintre valoarea Q pentru BUY si valoarea Q pentru HOLD in starile in care agentul este in cash. Daca diferenta este pozitiva, agentul prefera BUY. Daca diferenta este negativa, agentul prefera HOLD.



**Figura 8:** Diferenta dintre  $Q(\text{BUY})$  si  $Q(\text{HOLD})$  pentru starile cash, comparativ intre SPY si QQQ. Valorile pozitive indica preferinta pentru BUY, iar valorile negative indica preferinta pentru HOLD.

Graficul arata ca agentul nu copiaza simplu semnalul ML. Nu este doar o regula de tipul “cumpara cand ML spune crește”. Pe SPY, apare chiar un comportament oarecum ciudat la prima vedere: preferinta pentru BUY apare mai ales cand trendul este jos sau neutru, iar semnalul ML este negativ sau neutru. Cand semnalul ML este pozitiv, agentul prefera uneori



HOLD. Asta nu inseamna ca regula este intuitiva financiar, ci doar ca asa au iesit valorile Q pe datele de antrenare.

Pe QQQ, preferinta pentru BUY este puternica in starile cu trend jos si apare si pentru unele stari cu trend neutru. In schimb, pentru trend sus, diferenta este negativa in toate cele trei stari ML, deci agentul prefera sa astepte. Aceasta explica de ce politica invatata pe QQQ este mai selectiva decat o strategie Buy and Hold.

## 9.4 Interpretarea generala

Rezultatele nu sunt la fel pe SPY si QQQ. Pe SPY, sistemul ML + Q-learning depaseste usor Buy and Hold la randament final, dar sta aproape tot timpul investit si are drawdown similar. Deci nu pot spune ca agentul a redus riscul pe SPY. Mai degraba a ajuns la o strategie foarte apropiata de Buy and Hold, dar cu un rezultat final putin mai bun.

Pe QQQ, situatia este diferita. ML + Q-learning nu depaseste Buy and Hold la randament, dar are drawdown mai mic. Aici agentul pare mai selectiv si sta mai putin timp in piata. Asta reduce o parte din scaderi, dar pierde si o parte din cresterea pietei.

Concluzia trebuie formulata atent. Nu pot spune ca sistemul bate piata in general. Pot spune doar ca, pe aceasta impartire cronologica train/test, combinarea Logistic Regression cu Q-learning produce o politica functionala si usor de interpretat. Pe SPY rezultatul este competitiv cu Buy and Hold. Pe QQQ, strategia este mai defensiva, dar sacrifica randament.

## 10 State of the Art si pozitionarea proiectului

### 10.1 Context

Tranzactionarea algoritmica este o zona in care se folosesc atat metode simple, bazate pe reguli, cat si metode de Machine Learning si Reinforcement Learning. O strategie simpla poate folosi medii mobile sau praguri fixe. Problema este ca aceste reguli sunt rigide si pot merge bine intr-o perioada, dar prost in alta.

Machine Learning se foloseste de multe ori pentru predictie. Modelul invata o relatie intre datele de intrare si o eticheta. In proiectul meu, Logistic Regression incearca sa estimeze daca randamentul pe urmatoarele 5 zile este peste mediana din train. Modelul nu decide direct actiunea de trading, ci produce un semnal.

Reinforcement Learning este mai potrivit pentru partea de decizie. Agentul vede o stare, alege o actiune si primeste o recompensa, idee explicata si in cartea lui Sutton si Barto [4]. In trading, asta se potriveste natural cu intrebarea: cumpar, vand sau astept?

### 10.2 Lucrarea Theate si Ernst

O lucrare importanta pentru proiect este cea a lui Theate si Ernst, *An Application of Deep Reinforcement Learning to Algorithmic Trading* [5]. Autorii propun TDQN, o strategie bazata pe Deep Q-Network. Este o varianta mult mai avansata decat Q-learning-ul tabular folosit aici, deoarece functia Q este aproximata cu o retea neuronală.

Exista si lucrari de tip review care arata ca Deep Reinforcement Learning este folosit in trading, dar rezultatele trebuie citite atent, pentru ca depind mult de date, perioada si modul de evaluare [6]. Asta se vede si in proiectul meu: acelasi sistem se comporta diferit pe

SPY si QQQ.

### 10.3 Pozitionarea proiectului

Contributia proiectului consta in adaptarea ideii generale de agent RL pentru trading intr-o forma simplificata si interpretabila. In loc sa folosesc direct un model Deep Learning, am separat problema in doua componente: o componenta de predictie, care estimeaza directia pietei, si o componenta de decizie, care invata cand sa cumpere, sa vanda sau sa astepte.

Astfel, proiectul se pozitioneaza ca o demonstratie academica ML + RL. Avantajul principal este claritatea: se poate urmari direct cum datele devin feature-uri, cum modelul ML produce un semnal, cum acest semnal intra in starea Q-learning si cum politica finala este evaluata prin backtesting.

### 10.4 Comparatie orientativa cu TDQN

Dupa rezultate, am inclus si o comparatie orientativa cu lucrarea lui Theate si Ernst, unde este propus TDQN, un agent bazat pe Deep Q-Network pentru trading [5].

Comparatia nu este directa. Autorii folosesc Deep Reinforcement Learning, testeaza pe 30 de instrumente si au alta perioada de train/test: 2012–2017 pentru antrenare si 2018–2019 pentru testare. In proiectul meu sunt folosite doar SPY si QQQ, pe alta perioada, iar modelul este mai simplu.

Comparatia este un reper, proiectul nu depaseste TDQN. Totusi ideea generala este asemanatoare: un agent invata o politica de tranzactionare si apoi este evaluat prin backtesting.

**Tabelul 12:** Comparatie orientativa prin Sharpe Ratio

| Instrument | Theate B&H | Theate TDQN | Proiect B&H | Proiect ML+Q |
|------------|------------|-------------|-------------|--------------|
| SPY        | 0.834      | 0.834       | 0.68        | 0.72         |
| QQQ        | 0.845      | 0.845       | 0.83        | 0.84         |

Tabelul foloseste Sharpe Ratio doar ca reper orientativ. In lucrarea Theate si Ernst, pentru SPY si QQQ, TDQN are acelasi Sharpe Ratio ca Buy and Hold. In proiectul meu, ML + Q-learning obtine un Sharpe Ratio usor peste Buy and Hold pe aceasta impartire train/test, dar aceasta observatie nu poate fi generalizata. Nu inseamna ca modelul meu bate TDQN.

Pe QQQ se vede bine de ce Sharpe Ratio nu este suficient singur. ML + Q-learning are Sharpe foarte apropiat de Buy and Hold, dar randamentul final este mai mic. De aceea rezultatele trebuie citite impreuna: randament, drawdown, expunere si perioada testata.

**Tabelul 13:** Diferente metodologice si conceptuale intre TDQN si proiectul curent

| Aspect               | TDQN, Theate si Ernst                     | Proiectul curent                         |
|----------------------|-------------------------------------------|------------------------------------------|
| Model                | Deep Q-Network                            | Logistic Regression + Q-learning tabular |
| Functia Q            | Aproximata cu retea neuronală             | Salvata intr-o tabela Q                  |
| Reprezentarea starii | Mai bogata, prin model Deep Learning      | Discreta: semnal ML, pozitie, trend      |
| Date testate         | 30 de instrumente                         | 2 ETF-uri, SPY si QQQ                    |
| Perioada             | Train 2012–2017, test 2018–2019           | Train 2010–2016, test 2016–2018          |
| Actiuni              | Pozitii long si short                     | BUY, SELL, HOLD, fara short              |
| Interpretabilitate   | Mai greu de urmarit direct                | Mai usor de explicat prin Q-table        |
| Scop                 | Metoda mai avansata si evaluare mai larga | Demonstratie simpla si explicabila       |

Diferenta principala este modelul. TDQN foloseste o retea neuronală pentru functia Q, deci poate invata relatii mai complexe. In acest proiect, tabela Q are doar 18 stari. Din cauza asta este mai usor de explicat, dar este si mai limitata.

Prin urmare, comparatia cu TDQN este doar orientativa. Proiectul foloseste o idee similara, dar intr-o forma mult mai simpla. Rezultatele sunt utile pentru a demonstra pipeline-ul ML + RL, nu pentru a spune ca sistemul depaseste literatura existenta.

## 11 Limitari

Proiectul are cateva limitari destul de clare.

Prima limitare este ca am testat doar doua ETF-uri. SPY si QQQ sunt utile pentru experiment, dar nu sunt suficiente ca sa trag o concluzie generala despre piata. Pentru o evaluare mai serioasa ar trebui testate mai multe instrumente.

A doua limitare este impartirea train/test. Am folosit o singura impartire cronologica, ceea ce este corect pentru timp, dar tot ramane o singura perioada de test. O varianta mai buna ar fi walk-forward validation, unde modelul este antrenat si testat pe mai multe ferestre.

Feature-urile sunt simple. Ele vin doar din pret si volum: randamente, volum si medii mobile. Nu folosesc stiri, date macroeconomice sau sentiment de piata. Probabil ca modelul pierde multa informatie din cauza asta.

Si Q-learning-ul este simplificat. Are doar 18 stari, ceea ce il face usor de explicat, dar limiteaza mult ce poate invata. Un model mai avansat, de exemplu DQN, ar putea lucra cu o stare mai bogata, dar ar fi mai greu de justificat si de explicat in acest proiect.

Mai este si problema reward-ului. In varianta finala am folosit randamentul zilnic, fara limitare artificiala. Mi se pare o varianta mai apropiata de backtest, dar face agentul sensibil la zile cu miscari mari. Daca schimb recompensa, este posibil sa se schimbe si politica invatata.

Costul de tranzactionare este inclus, dar simplificat. In realitate ar exista si spread, slippage, taxe diferite si intarzieri de executie.

În final, rezultatele din 2016–2018 nu spun automat ce s-ar întâmpla în alta perioadă. Piața își poate schimba comportamentul, iar un model simplu se poate adapta greu.

## 12 Manual de utilizare

Aplicația se rulează din linia de comandă, din directorul proiectului. Înainte de rulare trebuie instalate bibliotecile necesare:

```
pip install -r requirements.txt
```

Pentru rularea experimentului principal se execută:

```
python main.py
```

Tickerul folosit se setează în fișierul `config.py`, prin modificarea variabilei `TICKER`. Pentru experimentul principal se folosește:

```
TICKER = "SPY"
```

Pentru verificarea suplimentară pe `QQQ` se folosește:

```
TICKER = "QQQ"
```

La rulare, aplicația afișează perioada de `train` și `test`, feature-urile folosite de Logistic Regression, acuratețea modelului, ponderile învățate, regulile pentru semnalul ML și tabelul final de benchmark. Tabelul compară strategiile după valoarea finală a portofoliului, randament, număr de tranzacții, expunere, Sharpe Ratio și Max Drawdown.

```

PS C:\Users\david\Desktop\ai-proiect-final> python main.py
ticker: SPY
randuri dupa curatare: 2240
train: 2010-02-01 -> 2016-04-22 (1568 randuri)
test: 2016-04-25 -> 2018-12-21 (672 randuri)

feature-uri LR:
- daily_return
- open_close_return
- high_low_range
- volume_change
- ma_ratio

LR:
target: future_return_5d > mediana train (0.004515)
baseline train/test: 50.00% / 54.61%
accuracy train/test: 53.57% / 58.33%
bias: 0.474098

ponderi LR:
daily_return_norm: -0.312674
open_close_return_norm: -0.227683
high_low_range_norm: 0.953636
volume_change_norm: 0.416079
ma_ratio_norm: -0.552194

ml_signal:
0 daca prob < 0.49
1 daca prob e intre 0.49 si 0.51
2 daca prob > 0.51

benchmark:

```

| strategie          | valoare finala | randament % | tranzactii | expunere % | Sharpe Ratio | Max Drawdown % |
|--------------------|----------------|-------------|------------|------------|--------------|----------------|
| buy and hold       | 121745.02      | 21.75       | 1.00       | 100.00     | 0.68         | 17.16          |
| LR only            | 90913.29       | -9.09       | 135.00     | 27.38      | -0.34        | 17.32          |
| random agent avg   | 89789.12       | -10.21      | 336.27     | 49.27      | -0.44        | 16.92          |
| ML + Q-learning    | 123192.36      | 23.19       | 1.00       | 99.11      | 0.72         | 17.16          |
| Q-learning fara ML | 111731.59      | 11.73       | 21.00      | 26.64      | 0.53         | 11.84          |

```

PS C:\Users\david\Desktop\ai-proiect-final>

```

**Figura 9:** Exemplu de rulare pentru tickerul SPY.

```

PS C:\Users\david\Desktop\ai-proiect-final> python main.py
ticker: QQQ
randuri dupa curatare: 2240
train: 2010-02-01 -> 2016-04-22 (1568 randuri)
test: 2016-04-25 -> 2018-12-21 (672 randuri)

feature-uri LR:
- daily_return
- open_close_return
- high_low_range
- volume_change
- ma_ratio

LR:
target: future_return_5d > mediana train (0.005606)
baseline train/test: 50.00% / 50.30%
accuracy train/test: 53.95% / 53.57%
bias: 0.577429

ponderi LR:
daily_return_norm: -0.331951
open_close_return_norm: -0.423075
high_low_range_norm: 0.720454
volume_change_norm: 0.158979
ma_ratio_norm: -0.433881

ml_signal:
0 daca prob < 0.49
1 daca prob e intre 0.49 si 0.51
2 daca prob > 0.51

benchmark:

```

| strategie          | valoare finala | randament % | tranzactii | expunere % | Sharpe Ratio | Max Drawdown % |
|--------------------|----------------|-------------|------------|------------|--------------|----------------|
| buy and hold       | 138454.97      | 38.45       | 1.00       | 100.00     | 0.83         | 20.83          |
| LR only            | 123339.50      | 23.34       | 107.00     | 33.93      | 0.69         | 13.21          |
| random agent avg   | 97113.95       | -2.89       | 336.27     | 49.27      | -0.06        | 18.81          |
| ML + Q-learning    | 131935.97      | 31.94       | 45.00      | 43.30      | 0.84         | 17.02          |
| Q-learning fara ML | 120557.67      | 20.56       | 31.00      | 29.76      | 0.63         | 17.47          |

```

PS C:\Users\david\Desktop\ai-proiect-final>

```

Figura 10: Exemplu de rulare pentru tickerul QQQ.

## 13 Concluzii

Proiectul a construit un sistem complet care combina Logistic Regression si Q-learning. Logistic Regression produce un semnal despre directia pietei, iar Q-learning foloseste acest semnal ca parte din starea agentului.

Rezultatele au fost mixte. Pe SPY, ML + Q-learning obtine 23.19%, fata de 21.75% pentru Buy and Hold. Este un rezultat bun, dar trebuie spus ca strategia sta aproape tot timpul investita, deci este destul de apropiata de Buy and Hold. Pe QQQ, ML + Q-learning obtine 31.94%, sub Buy and Hold, care obtine 38.45%. In schimb, pe QQQ drawdown-ul este mai mic.

Pentru mine, rezultatul important nu este ca sistemul ar fi o strategie reala de investitii. Nu ar fi corect sa spun asta dupa doar doua ETF-uri si o singura perioada de test. Rezultatul important este ca pipeline-ul functioneaza: datele sunt prelucrate, modelul ML produce un semnal, agentul RL foloseste semnalul, iar strategia este evaluata prin backtesting.

Comparatia cu strategiile simple ajuta la interpretare. Logistic Regression folosit direct este mai slab decat sistemul complet, iar Q-learning fara ML ramane sub ML + Q-learning pe ambele ETF-uri. Asta sugereaza ca semnalul ML adauga informatie utila, chiar daca nu garanteaza depasirea Buy and Hold.

Comparatia cu TDQN arata si limita proiectului. Ideea generala este asemanatoare, dar implementarea este mult mai simpla. Din acest motiv, proiectul trebuie vazut ca o demonstratie academica ML + RL, nu ca o metoda State of the Art.

Ca imbunatatiri viitoare, as testa mai multe perioade prin walk-forward validation, mai multe instrumente financiare, feature-uri mai bogate si eventual o varianta cu Deep Q-learning.

## Bibliografie

- [1] Guillem Servera. *S&P 500 and NASDAQ 100: Daily Data*. Kaggle Dataset. Disponibil online: <https://www.kaggle.com/datasets/guillemservera/sp500-nasdaq-spy-qq-ohlcv-data>
- [2] State Street Global Advisors. *SPDR S&P 500 ETF Trust*. Disponibil online: <https://www.ssga.com/us/en/intermediary/etfs/spdr-sp-500-etf-trust-spy>
- [3] Invesco. *Invesco QQQ ETF*. Disponibil online: <https://www.invesco.com/qqq-etf/en/home.html>
- [4] Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. Second Edition, MIT Press, 2018.
- [5] Theate, T. and Ernst, D. *An Application of Deep Reinforcement Learning to Algorithmic Trading*. Expert Systems with Applications, 173, 114632, 2021.
- [6] Pricope, T.-V. *Deep Reinforcement Learning in Quantitative Algorithmic Trading: A Review*. arXiv:2106.00123, 2021.